Technical Test: Bioinformatics Engineer Intern

Objective

Build a simple REST API for DNA sequence analysis: (1) alignment to a reference with variant detection, and (2) translation to protein. This test assesses backend skills, basic bioinformatics understanding, and engineering quality (git, testing, documentation).

General Requirements

- Language: Python.
- Git is required, with a meaningful commit history (not a single large commit).
- Submit your work as a GitHub/GitLab repository (public).
- Time allowed: 3 days.
- Use the supplied reference fasta (`REF_001`, a 300 bp synthetic coding sequence). Treat the supplied sequences as the forward strand of the same complete fragment. The reference coding frame starts at base 1.
- Bioinformatics libraries and Python web frameworks are allowed. The five patient FASTA files are sample inputs; the API must also handle other inputs that meet the rules below.

Shared FASTA validation

- Both endpoints must accept exactly one FASTA record, with a non-empty header and a sequence of 1 to 1,000 nucleotides after removing line breaks and surrounding whitespace.
- Accept A, C, G, T, and N only, case-insensitively, and convert the sequence to uppercase. Multiline sequences and blank lines are allowed. Reject invalid characters, embedded sequence whitespace, empty sequences, multiple records, and files exceeding the length limit.
- Set `sequence_id` to the first whitespace-delimited token after `>` in the FASTA header. For the supplied reference, `reference_id` is `REF_001`.
- Missing file, malformed FASTA, and invalid frame values must return a clear JSON error response (HTTP 400 or 422). N represents an unknown base and is valid input; do not reject a sequence solely because it contains many Ns.

1. Alignment and Variant Detection

`POST /api/align`

Request format

Send multipart/form-data with one uploaded FASTA file in the field file. The reference is loaded by the server, not supplied in the request.

```
curl -X POST http://localhost:8000/api/align \
  -F "file=@patient_02.fasta"
```

The example response below uses the complete supplied patient_02.fasta.

Example successful response

HTTP 200. The aligned strings are shortened here with … for readability; your API must return both complete aligned strings. For this example only, scoring is match +1, mismatch -1, and -2 per gap character.

```
{
  "sequence_id": "patient_02",
  "reference_id": "REF_001",
  "alignment": {
      "score": 294.0,
      "identity": 0.99,
      "reference_aligned": "ATGAGTTCG...",
      "query_aligned": "ATGAGTTCG..."
  },
  "variants": [
      {"position": 52, "ref": "C", "alt": "A", "type": "SNV"},
      {"position": 177, "ref": "A", "alt": "T", "type": "SNV"},
      {"position": 245, "ref": "T", "alt": "G", "type": "SNV"}
  ]
}
```

Alignment and variant rules

- Validate input using the shared FASTA rules.
- Use global alignment. Document match, mismatch, gap, and N scoring in the README. Return the score; it may differ from the example when scoring differs.
- Use type values "SNV", "insertion", and "deletion". If no variants are found, return variants: [].
- SNV and deletion positions are 1-based reference coordinates. For deletions, position is the first deleted base; ref contains the deleted sequence and alt is "-".
- For insertions, position is the preceding reference base, ref is "-", and alt contains the inserted sequence. Position 7 means after reference base 7; position 0 means before base 1.
- Group consecutive inserted or deleted bases into one event. Sort events by position. VCF output and normalization are not required.

- Call SNVs only between differing aligned A/C/G/T bases; N is not an SNV. Gap events may be reported in N-containing regions with documented limitations.
- identity = identical A/C/G/T columns / all alignment columns, including gaps and N. Return a value from 0 to 1; N/N is not a match.
- For tied optimal alignments, choose consistently and document the rule. Equivalent indel placements are acceptable if supported by the returned alignment.

2. Protein Translation

`POST /api/translate`

Request format

Send multipart/form-data with the FASTA file in the field file and an optional integer field frame (1, 2, or 3; default 1).

```
curl -X POST http://localhost:8000/api/translate \
  -F "file=@patient_02.fasta" -F "frame=1"
```

The example below uses the complete supplied patient_02.fasta and stops at its first stop codon.

Example successful response

HTTP 200. The protein string below is complete; length excludes the stop codon.


```
{
  "sequence_id": "patient_02",
  "frame": 1,
  "protein":
"MSSFVPLLYVSRVGTSNRLFSLLTKFTWVSVTRLRNPAEFAACVRHPYLAHSSQSHEHNLALVITSGQSTPDSKGEQN
WVS",
  "length": 81,
  "has_stop_codon": true
}
```


Translation rules

- Use the standard genetic code. Frame 1 starts at nucleotide 1, frame 2 at nucleotide 2, and frame 3 at nucleotide 3. Translate the supplied strand directly, without searching for a start codon or reverse complementing the sequence.
- After applying the frame offset, translate complete codons only. Ignore any trailing 1 or 2 nucleotides. Translate every codon containing N as X, even if its amino acid could otherwise be inferred.
- Stop before the first unambiguous stop codon (TAA, TAG, or TGA). Do not include * in protein. Set has_stop_codon to true if a stop is encountered; otherwise false. Set length to len(protein), including X characters.

- Apply the shared FASTA validation rules. Reject frame values outside 1-3. If the selected frame leaves no complete codon, return protein "", length 0, and has_stop_codon false.

Engineering Requirements

- Clean project structure (router, service/logic, and schema kept separate).
- Appropriate HTTP status codes and error messages (400/422 for invalid input).
- Provide requirements.txt with the dependencies needed to run the API and tests, and installation instructions in the README.
- README: installation, how to run the API, example curl requests for both endpoints, how to run tests, alignment scoring, identity and variant conventions, assumptions, and limitations.
- Include automated tests for both endpoints: the supplied patient files, invalid and multiple-record FASTA, lowercase input, N-containing codons, each reading frame, stop handling, incomplete final codons, and the sequence length limit. Verify variant coordinates and grouping, not just response status.

Bonus (optional)

- Build a simple web interface connected to the API, allowing users to upload a FASTA file, run alignment or translation, select a reading frame for translation, and view the results and error messages. Any frontend framework or plain HTML/CSS/JavaScript is acceptable.
- Dockerfile.
- A combined endpoint POST /api/analyze that aligns and translates the query and annotates variant effects using reference frame 1: synonymous, missense, nonsense, frameshift, or in-frame insertion/deletion. State how ambiguous codons and multiple nearby variants are handled; an undetermined result is acceptable when documented.
- Implement global alignment (for example, Needleman-Wunsch) and/or translation yourself without bioinformatics libraries.

Bonus items are optional. Complete the two required endpoints and their tests first.

Deliverables

- Link to the GitHub/GitLab repository.
- A complete README.
- Automated test results (terminal output in the README or a screenshot), together with the test code and a reproducible command to run it.